

Information Flow Security

Logan Green

What is Information Flow?

- How data moves throughout a program

What is Information Flow?

- How data moves throughout a program
- Security issue when secret data reaches public places

What is Information Flow?

- How data moves throughout a program
- Security issue when secret data reaches public places

```
var l, h  
l := h
```

Explicit Flows

What is Information Flow?

- How data moves throughout a program
- Security issue when secret data reaches public places

```
var l, h  
l := h
```

Explicit Flows

```
var l, h  
if h = true then  
    l = 0  
else  
    l = 1
```

Side Channel (Implicit Flow)

Motivation

- Access control alone is not enough

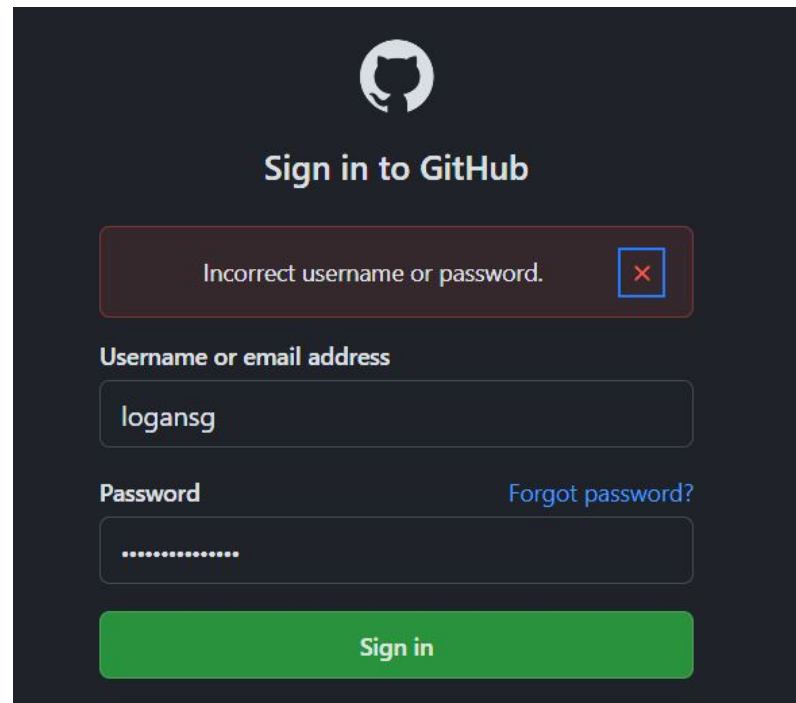


Motivation

- Access control alone is not enough
- Sensitive data can leak during computation

Motivation

- Access control alone is not enough
- Sensitive data can leak during computation
- Sometimes the release of secret data must be allowed
 - Password checks
 - Average Salary Data



The image shows the GitHub login interface. At the top is the GitHub logo. Below it is the text "Sign in to GitHub". There is a red error message box that says "Incorrect username or password." with a red 'x' icon. Below the error message are two input fields: "Username or email address" containing the text "logansg" and "Password" containing a series of dots. To the right of the password field is a link that says "Forgot password?". At the bottom is a green button that says "Sign in".

Roadmap

- Foundations
- Non-Interference
- Dependency
- Practical Enforcement
- Declassification
- Modern reconstruction

Denning (1976)

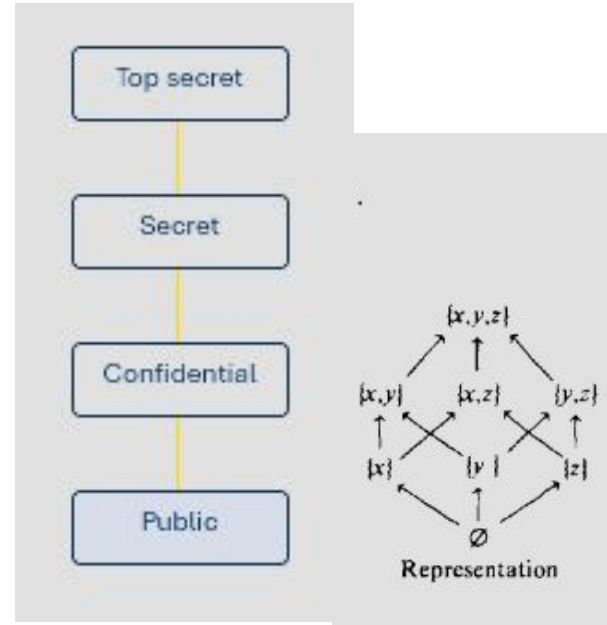
- Access control misses indirect leaks through computation or control

Denning (1976)

- Access control misses indirect leaks through computation or control
- Model security classes and permitted flows as a bounded lattice

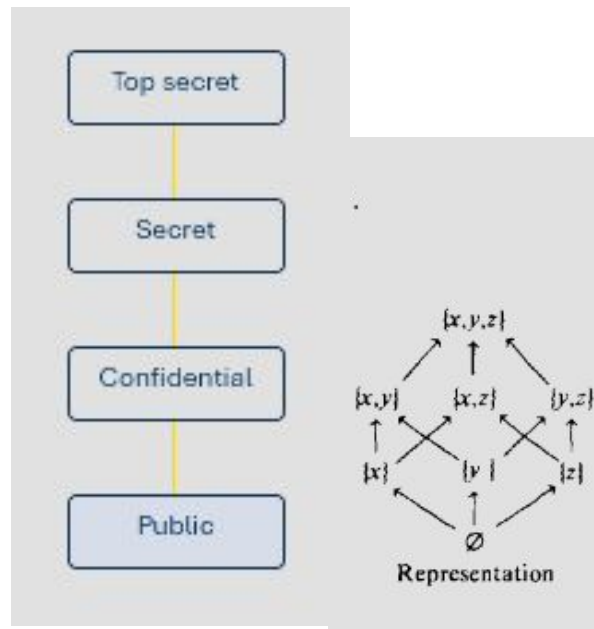
Why a lattice?

- Ordering
 - Classes need a consistent “can flow to” order



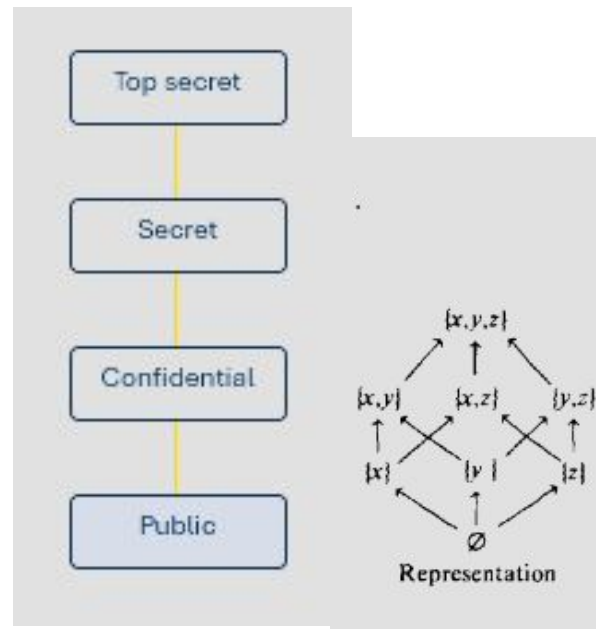
Why a lattice?

- Ordering
 - Classes need a consistent “can flow to” order
- Combining Influences
 - Programs combine data
 - We need to combine labels $\oplus$



Why a lattice?

- Ordering
 - Classes need a consistent “can flow to” order
- Combining Influences
 - Programs combine data
 - We need to combine labels $\oplus$
- Partial Order
 - Reflexive: $A \rightarrow A$
 - Transitive $A \rightarrow B, B \rightarrow C, A \rightarrow C$
 - Antisymmetric $A \rightarrow B, B \rightarrow A, A = B$



Denning (1976)

- Access control misses indirect leaks through computation or control
- Model security classes and permitted flows as a bounded lattice
- Basis for certification and run-time enforcement

Verification example

Public = Top secret + Top secret



Verification example

Public = Top secret + Top secret

Top secret $\oplus$ Top secret = Top Secret



Verification example

Public = Top secret + Top secret

Top secret $\oplus$ Top secret = Top Secret

WAIT!



Verification example

Public = Top secret + Top secret

Top secret $\oplus$ Top secret = Top Secret

WAIT!

Top secret is not allowed to flow into Public!!!



Denning (1976)

- Access control misses indirect leaks through computation or control
- Model security classes and permitted flows as a bounded lattice
- Basis for certification and run-time enforcement
- Does not handle covert channels or every system-level security issue

Non-Interference

- Changing only the secret input should not change what a low observer can see



Fix low, vary high → observable behavior stays the same

Goguen & Meseguer (1982)

- Earlier security work often mixed

Goguen & Meseguer (1982)

- Earlier security work often mixed
 - The rules we want

Goguen & Meseguer (1982)

- Earlier security work often mixed
 - The rules we want
 - The system we built

Goguen & Meseguer (1982)

- Earlier security work often mixed
 - The rules we want
 - The system we built
- This paper argues that these should be separated
 - Security policy = set of non-interference rules

Goguen & Meseguer (1982)

- Earlier security work often mixed
 - The rules we want
 - The system we built
- This paper argues that these should be separated
 - Security policy = set of non-interference rules
 - System model = machine describing how the system behaves
 - $S \times U \times C \rightarrow S$
 - $S \times U \rightarrow \text{out}$

Goguen & Meseguer (1982)

- Earlier security work often mixed
 - The rules we want
 - The system we built
- This paper argues that these should be separated
 - Security policy = set of non-interference rules
 - System model = machine describing how the system behaves
 - $S \times U \times C \rightarrow S$
 - $S \times U \rightarrow \text{out}$
- Verification becomes proving the model satisfies the policy

Goguen & Meseguer (1982)

- Earlier security work often mixed
 - The rules we want
 - The system we built
- This paper argues that these should be separated
 - Security policy = set of non-interference rules
 - System model = machine describing how the system behaves
 - $S \times U \times C \rightarrow S$
 - $S \times U \rightarrow \text{out}$
- Verification becomes proving the model satisfies the policy
- Does not address authentication, inference attacks, aggregation, or fault tolerance

Dependency

- An output depends on an input if changing the input could change the output

Dependency

- An output depends on an input if changing the input could change the output
 - $z = x + y$
 - z depends on x and y

Dependency

- An output depends on an input if changing the input could change the output
 - $z = x + y$
 - z depends on x and y
 - If $x == \text{true}$: $y = 1$; else: $y = 2$;
 - y depends on x

Dependency

- An output depends on an input if changing the input could change the output
 - $z = x + y$
 - z depends on x and y
 - If $x == \text{true}$: $y = 1$; else: $y = 2$;
 - y depends on x
- In security, public outputs should not depend on secret input

Abadi et al. (1999)

- Dependency shows up in more than just security

Abadi et al. (1999)

- Dependency shows up in more than just security
 - Secure information flow
 - Program slicing
 - Binding-time analysis
 - Call tracking

Abadi et al. (1999)

- Dependency shows up in more than just security
 - Secure information flow
 - Program slicing
 - Binding-time analysis
 - Call tracking
- Do these share one common core idea?

Abadi et al. (1999)

- Dependency shows up in more than just security
 - Secure information flow
 - Program slicing
 - Binding-time analysis
 - Call tracking
- Do these share one common core idea?
- Dependency Core Calculus (DCC)
 - Small core language for reasoning about dependency

Abadi et al. (1999)

- Dependency shows up in more than just security
 - Secure information flow
 - Program slicing
 - Binding-time analysis
 - Call tracking
- Do these share one common core idea?
- Dependency Core Calculus (DCC)
 - Small core language for reasoning about dependency
- Paper translates existing calculi into DCC

Abadi et al. (1999)

- Define DCC formally and give it a semantic model
- Show that other analysis translate into DCC

Abadi et al. (1999)

- Define DCC formally and give it a semantic model
- Show that other analysis translate into DCC
- Why does this matter?

Abadi et al. (1999)

- Define DCC formally and give it a semantic model
- Show that other analysis translate into DCC
- Why does this matter?
 - Compare different dependency analyses
 - Helps reveal gaps or odd choices in earlier systems
 - Get simpler proofs

Abadi et al. (1999)

- Define DCC formally and give it a semantic model
- Show that other analysis translate into DCC
- Why does this matter?
 - Compare different dependency analyses
 - Helps reveal gaps or odd choices in earlier systems
 - Get simpler proofs
- Not every dependency analysis fits into DCC

Abadi et al. (1999)

- Define DCC formally and give it a semantic model
- Show that other analysis translate into DCC
- Why does this matter?
 - Compare different dependency analyses
 - Helps reveal gaps or odd choices in earlier systems
 - Get simpler proofs
- Not every dependency analysis fits into DCC
 - DCC says changing secret input should not change observable output or termination time

Abadi et al. (1999)

- Define DCC formally and give it a semantic model
- Show that other analysis translate into DCC
- Why does this matter?
 - Compare different dependency analyses
 - Helps reveal gaps or odd choices in earlier systems
 - Get simpler proofs
- Not every dependency analysis fits into DCC
 - DCC says changing secret input should not change observable output or termination time
 - Call-By-Value can change termination timing

Myers (1999)

- Earlier information flow languages enforced flow correctly, but too restrictive

Myers (1999)

- Earlier information flow languages enforced flow correctly, but too restrictive
- JFlow extends Java with mostly static information flow checking

Myers (1999)

- Earlier information flow languages enforced flow correctly, but too restrictive
- JFlow extends Java with mostly static information flow checking
- Adds decentralized labels and safe declassification
 - {Alice: Bob, Logan}, {James: Alice, Bob}

Myers (1999)

- Earlier information flow languages enforced flow correctly, but too restrictive
- JFlow extends Java with mostly static information flow checking
- Adds decentralized labels and safe declassification
 - {Alice: Bob, Logan}, {James: Alice, Bob}
- Adds polymorphism, run-time label checks, and label inference

Myers (1999)

- Earlier information flow languages enforced flow correctly, but too restrictive
- JFlow extends Java with mostly static information flow checking
- Adds decentralized labels and safe declassification
 - {Alice: Bob, Logan}, {James: Alice, Bob}
- Adds polymorphism, run-time label checks, and label inference
- Supports features: objects, subclassing, access control, exceptions

Myers (1999)

```
int {public} x;  
boolean {secret} b;  
...  
int x = 0;  
if (b) {  
    x = 1;  
}
```



Myers (1999)

```
label {L} lb;  
int {*lb} x;  
int {p:} y;  
switch label(x) {  
    case (int {y} z) y = z;  
    else throw new UnsafeTransfer();  
}
```

Myers (1999)

- Earlier information flow languages enforced flow correctly, but too restrictive
- JFlow extends Java with mostly static information flow checking
- Adds decentralized labels and safe declassification
 - {Alice: Bob, Logan}, {James: Alice, Bob}
- Adds polymorphism, run-time label checks, and label inference
- Supports features: objects, subclassing, access control, exceptions
- Define language, give formal checking rules, worked examples, compiler

Myers (1999)

- Earlier information flow languages enforced flow correctly, but too restrictive
- JFlow extends Java with mostly static information flow checking
- Adds decentralized labels and safe declassification
 - {Alice: Bob, Logan}, {James: Alice, Bob}
- Adds polymorphism, run-time label checks, and label inference
- Supports features: objects, subclassing, access control, exceptions
- Define language, give formal checking rules, worked examples, compiler
- Does not address covert channels, omits some features

Sabelfeld & Sands (2005)

- Useful systems must release some secret info - declassification

Sabelfeld & Sands (2005)

- Useful systems must release some secret info
- Declassification research has become fragmented

Sabelfeld & Sands (2005)

- Useful systems must release some secret info
- Declassification research has become fragmented
- Organizes release policies along four axes

Sabelfeld & Sands (2005)

- Useful systems must release some secret info
- Declassification research has become fragmented
- Organizes release policies along four axes
 - **What** information is released

Sabelfeld & Sands (2005)

- Useful systems must release some secret info
- Declassification research has become fragmented
- Organizes release policies along four axes
 - **What** information is released
 - **Who** is allowed to release it

Sabelfeld & Sands (2005)

- Useful systems must release some secret info
- Declassification research has become fragmented
- Organizes release policies along four axes
 - **What** information is released
 - **Who** is allowed to release it
 - **Where** release happens in the system

Sabelfeld & Sands (2005)

- Useful systems must release some secret info
- Declassification research has become fragmented
- Organizes release policies along four axes
 - **What** information is released
 - **Who** is allowed to release it
 - **Where** release happens in the system
 - **When** release is allowed

Sabelfeld & Sands (2005)

- Useful systems must release some secret info
- Declassification research has become fragmented
- Organizes release policies along four axes
 - **What** information is released
 - **Who** is allowed to release it
 - **Where** release happens in the system
 - **When** release is allowed
- Proposes semantic sanity checks for declassification

Sabelfeld & Sands (2005)

- Useful systems must release some secret info
- Declassification research has become fragmented
- Organizes release policies along four axes
 - **What** information is released
 - **Who** is allowed to release it
 - **Where** release happens in the system
 - **When** release is allowed
- Proposes semantic sanity checks for declassification
 - Semantic consistency

Sabelfeld & Sands (2005)

- Useful systems must release some secret info
- Declassification research has become fragmented
- Organizes release policies along four axes
 - **What** information is released
 - **Who** is allowed to release it
 - **Where** release happens in the system
 - **When** release is allowed
- Proposes semantic sanity checks for declassification
 - Semantic consistency
 - Conservativity



Sabelfeld & Sands (2005)

- Useful systems must release some secret info
- Declassification research has become fragmented
- Organizes release policies along four axes
 - **What** information is released
 - **Who** is allowed to release it
 - **Where** release happens in the system
 - **When** release is allowed
- Proposes semantic sanity checks for declassification
 - Semantic consistency
 - Conservativity
 - Monotonicity of release



Sabelfeld & Sands (2005)

- Useful systems must release some secret info
- Declassification research has become fragmented
- Organizes release policies along four axes
 - **What** information is released
 - **Who** is allowed to release it
 - **Where** release happens in the system
 - **When** release is allowed
- Proposes semantic sanity checks for declassification
 - Semantic consistency
 - Conservativity
 - Monotonicity of release
 - Non-occlusion



Sabelfeld & Sands (2005)

What				
Property	Semantic consistency	Conservativity	Monotonicity of release	Non-occlusion
Partial release [Coh78, JL00, SS01, GM04, GM05]	✓	✓	N/A	✓
Delimited release [SM04]	✓	✓	✓	✓
Relaxed noninterference [LZ05]	×	✓	✓	✓
Naive release	✓	✓	✓	×
Who				
Robust declassification [MSZ04a]	✓*	✓	✓	✓
Qualified robust declassification [MSZ04a]	✓*	✓	×	×
Where				
Intransitive noninterference [MS04]	✓*	✓	×	✓
When				
Admissibility [DG00, GD03]	×	✓	×	✓
Noninterference “until” [CM04]	×	×	✓	✓
Typeless noninterference “until”	✓*	✓	×	×

* Semantic anomalies

Figure 1. Checking principles of declassification.

Sabelfeld & Sands (2005)

- Useful systems must release some secret info
- Declassification research has become fragmented
- Organizes release policies along four axes
 - **What** information is released
 - **Who** is allowed to release it
 - **Where** release happens in the system
 - **When** release is allowed
- Proposes semantic sanity checks for declassification
 - Semantic consistency
 - Conservativity
 - Monotonicity of release
 - Non-occlusion
- Paper is just a roadmap, doesn't give an answer that solves declassification



- Programmers express information flow properties with parametric polymorphism

- Programmers express information flow properties with parametric polymorphism

$\text{id} : a \rightarrow a$

- Programmers express information flow properties with parametric polymorphism

$\text{id} : a \rightarrow a$

$\text{second} : a \rightarrow B \rightarrow B$

- Programmers express information flow properties with parametric polymorphism

$\text{id} : a \rightarrow a$

$\text{second} : a \rightarrow B \rightarrow B$

$\text{map} : (a \rightarrow B) \rightarrow \text{list } a \rightarrow \text{list } B$

- Programmers express information flow properties with parametric polymorphism

$\text{id} : a \rightarrow a$

$\text{second} : a \rightarrow B \rightarrow B$

$\text{map} : (a \rightarrow B) \rightarrow \text{list } a \rightarrow \text{list } B$

- But what if we don't want abstract types?

- How do we track something non-polymorphic

`incr : int -> int`

- How do we track something non-polymorphic

`incr : int -> int`

- Separate data abstraction from information flow

- How do we track something non-polymorphic

`incr : int -> int`

- Separate data abstraction from information flow
- Tag types with *Dependency Variables*

- How do we track something non-polymorphic

$\text{incr} : \text{int} \rightarrow \text{int}$

- Separate data abstraction from information flow
- Tag types with *Dependency Variables*

$\text{incr} : \alpha \text{ int} \rightarrow \alpha \text{ int}$

Dependency Sets

$\text{add} : A \text{ int} \rightarrow B \text{ int} \rightarrow ?$

- Currently only 1 dependency per type

Dependency Sets

$\text{add} : a \text{ int} -> B \text{ int} -> ?$

- Currently only 1 dependency per type
- Dependency sets!

Dependency Sets

$\text{add} : \alpha \text{ int} \rightarrow B \text{ int} \rightarrow ?$

- Currently only 1 dependency per type
- Dependency sets!

$\text{add} : [\alpha] \text{ int} \rightarrow [B] \text{ int} \rightarrow [\alpha B] \text{ int}$

Password Check Example

```
let pass : [ pwd ] string = "password123!"
```

```
let check : [  $\alpha$  ] string -> [  $\alpha$  pwd ] bool  
    = fun attempt -> attempt == pass
```

Password Check Example

```
let pass : [ pwd ] string = "password123!"
```

```
let check : [ a ] string -> [ a pwd ] bool  
    = fun attempt -> attempt == pass
```

- Too conservative for the real world
- If the resulting bool is dependant on the password it shouldn't be released

Declassification

```
let check : [  $\alpha$  ] string -> [  $\alpha$  pwd ] bool  
  = fun attempt -> attempt == declassify(pass)
```

- Subverts the type system
- Basically an unsafe cast

Declassification

```
let check : [  $\alpha$  ] string -> [  $\alpha$  pwd ] bool  
  = fun attempt -> attempt == declassify(pass)
```

- Subverts the type system
- Basically an unsafe cast
- Can you get declassification without adding anything?

Existentials (Client View)

signature Queue = sig
 type t

enqueue : int -> t -> t

dequeue : t -> t * int

end



Existentials (Implementation View)

signature Queue = sig

 type t = List int

 enqueue : int -> List int -> List int

 dequeue : List int -> List int * int

end



Password Checker Module

```
structure pc : PasswordChecker = struct  
  dependency pwd
```

```
  pass : [ pwd ] string
```

```
  check : [  $\alpha$  ] string -> [  $\alpha$  ] bool
```

```
end
```

Password Checker Module

```
structure pc : PasswordChecker = struct  
  dependency pwd = [ ]
```

```
  pass : [ pwd ] string
```

```
  check : [ a ] string -> [ a ] bool
```

```
end
```

Password Checker Module

```
structure pc : PasswordChecker = struct
  dependency pwd = [ ]

  pass : [pwd] string = "password123"
  check : [a] string -> [a] bool
    = fun attempt -> attempt == pass

end
```

- Evaluation
 - Worked examples
 - Formal non-interference proof

Gouni et al. (2025)

- Evaluation
 - Worked examples
 - Formal non-interference proof
- No implementation (theoretical)

- Evaluation
 - Worked examples
 - Formal non-interference proof
- No implementation (theoretical)
- Extension to systems like ordinary type polymorphism left for future work

Open Questions in Information Flow

- Can you verify security without heavy annotational burden?

Open Questions in Information Flow

- Can you verify security without heavy annotational burden?
- What is the best policy for declassification?

Open Questions in Information Flow

- Can you verify security without heavy annotational burden?
- What is the best policy for declassification?
- How should IFC handle side channels?
 - Timing
 - Concurrency
 - Resource Usage

Key Takeaways

- Denning: lattice foundation for secure flow
- Goguen & Meseguer: noninterference + policy/model separation
- Abadi et al.: dependency as a unifying core
- Myers: practical mostly-static enforcement in Java
- Sabelfeld & Sands: declassification framework
- Gouni et al.: reconstruction of declassification from polymorphism/dependency

Thank you!

:)